VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
CITY school of Computer science and Engineering

PROJECT REPORT

E- COMMERCE WEBSITE

For WEB APPLICATION DEVELOPMENT COURSE

By Assoc.Prof Nguyen Van Sinh

Team member

Nguyễn Nhật Nam – ITITWE22149

Nguyễn Quang Thái – ITCSIU22227

Ngô Minh Ý – ITCSIU22169

Table of Contents

Abstract 5

I.Introduction 4

1. Overview 4

2. Problem Statement 5

3. Goal 5

4. Tools 5

1.1.1. GitHub 5

1.1.2. MySQL (Database Layer) 6

1.1.3. Redis 6

1.1.4. Swagger 7

II.Methodology 8

2.1 Sequence Diagrams 8

2.2. Backend Implementation Overview 15

2.2.1 Project Structure 16

2.3. Front-end Overview and Architecture 23

2.3.1 Project Structure 24

2.3.2 API Calls 27

2.3.2. State Management 28

2.3.3. Pages and Components 29

2.3.4 Features using extra libraries 30

2.5. Database Overview 31

2.5.1. Core Entities 31

2.5.2 Relationship Description 31

2.5.3 Entity Relationship Diagram 32

III.Demo 33

1. Authentication 33

2. Product Browsing 35

2.1 Home Page 35

2.2. Search 35

2.3 Shop 36

3. Shopping and Checkout 37

3.1 Shopping Cart 37

3.2. Checkout **38**
3.3 Order History **40**
4. Order & System Management **41**
4.1 User Account Management **41**
4.2 Admin / Staff Management **43**
IV. Discussion **44**
V. Conclusion **45**
VI. References **46**

Table of Figures

Figure 1.ERD **6**
Figure 2:Backend API Documentation **7**
Figure 3:User Registration **8**
Figure 4:User Login **9**
Figure 5:Browse Product **10**
Figure 6:Place Order **11**
Figure 7:Real-time chat **12**
Figure 8:Payment Process **13**
Figure 9:View Order **14**
Figure 10:Review **15**
Figure 11:Backend flow **16**
Figure 12:Backend project folder structure **17**
Figure 13:Order router mapping admin API endpoints to controllers. **18**
Figure 14:Create Order controller **18**
Figure 15:. Payment Service **19**
Figure 16:. Create Order Model **20**
Figure 17: Authentication and authorization middleware for securing API endpoints **21**
Figure 18:Authentication request validation schema **22**
Figure 19: JWT utility functions for token generation and verification **22**
Figure 20:Realtime module handling WebSocket-based communication **23**
Figure 21: Front-end architecture overview **24**
Figure 22: Front-end project folder structure. **26**
Figure 23: Example of API call with Axios. **28**
Figure 24:Global authentication state management on the front-end. **29**
Figure 25: Product detail page **30**
Figure 26: Entity Relationship Diagram (ERD) of the E-Commerce System **32**
Figure 27:Sign Up Form **33**
Figure 28: Sign Up Form **34**
Figure 29: Stay Home Page **35**

Figure 30: Search Function in Header **36**
Figure 31: Product Detail Page **37**
Figure 32: Add to Cart Action **37**
Figure 33: Shopping Cart Page **38**
Figure 34: Checkout Page **39**
Figure 35: Bank Transfer Payment Page **40**
Figure 36:Payment Successful Page **40**
Figure 37:Order History Page **41**
Figure 38: Order Detail Page **41**
Figure 39: User Account Menu **42**
Figure 40: My Account Page **42**
Figure 41: Admin Account Menu **43**
Figure 42: Admin Dashboard **43**
Figure 43: Staff Account Menu (Role-Based Access) **44**
Figure 44: Staff Panel **44**

Name	ID	Contribution
Nguyễn Nhật Nam	ITITWE22149	35%
Nguyễn Quang Thái	ITCSIU22227	35%
Ngô Minh Ý	ITCSIU22169	30%

Abstract

This study describes the design and implementation of a full-stack e-commerce web application utilizing a service-oriented architecture. The backend is developed using Express.js and TypeScript to provide a robust and scalable API, while the frontend is implemented with React and Vite to deliver a responsive and user-friendly interface. The system supports comprehensive functionalities for both customers and administrators, including product management, order processing, and user authentication. The proposed application aims to enhance the efficiency of online shopping processes and administrative operations, thereby providing a reliable and scalable solution for modern e-commerce platforms.

I.Introduction

1. Overview

This project is a full-stack e-commerce web application designed to provide a complete online shopping experience. The platform consists of a backend REST API built with Node.js, Express.js, and TypeScript, paired with a

modern frontend developed using React and Vite. The system supports three user roles: customers who can browse products, place orders, and make payments; staff members who manage orders and provide customer support; and administrators who have full control over products, categories, users, and system settings. Key features include user authentication with JWT, product catalog with search and filtering, shopping cart, multiple payment gateways (VNPay, MoMo, ZaloPay, COD), order management, product reviews, discount coupons, and real-time customer support chat using Socket.IO.

2. Problem Statement

Traditional retail businesses face significant challenges in reaching customers beyond their physical locations, resulting in limited market reach and reduced sales potential. Manual inventory management, order processing, and customer service operations are time-consuming, error-prone, and difficult to scale. Additionally, customers today expect convenient 24/7 shopping experiences with multiple payment options, real-time order tracking, and instant customer support. Small and medium businesses often lack the technical resources to build and maintain sophisticated e-commerce systems that meet these modern consumer expectations while ensuring security, performance, and reliability.

3. Goal

The primary goal of this project is to develop a scalable, secure, and user-friendly e-commerce platform that addresses the challenges faced by both businesses and customers in online retail. For businesses, the system aims to streamline product management, automate order processing, provide sales analytics, and reduce operational overhead. For customers, the goal is to deliver a seamless shopping experience with intuitive product discovery, secure checkout, flexible payment options, and responsive customer support. The platform is designed with modern web technologies and best practices to ensure high performance, maintainability, and the ability to scale as the business grows.

4. Tools

1.1.1. GitHub

Github repository of the project:
https://github.com/ngthais0607/e-commerce

1.1.2. MySQL (Database Layer)

The project used MySQL as the primary relational database to store application data such as users, products,categories, orders, and order items. MySQL ensures data consistency through relational constraints (primary keys, foreign keys) and supports efficient querying for features like search, filtering, and order management.

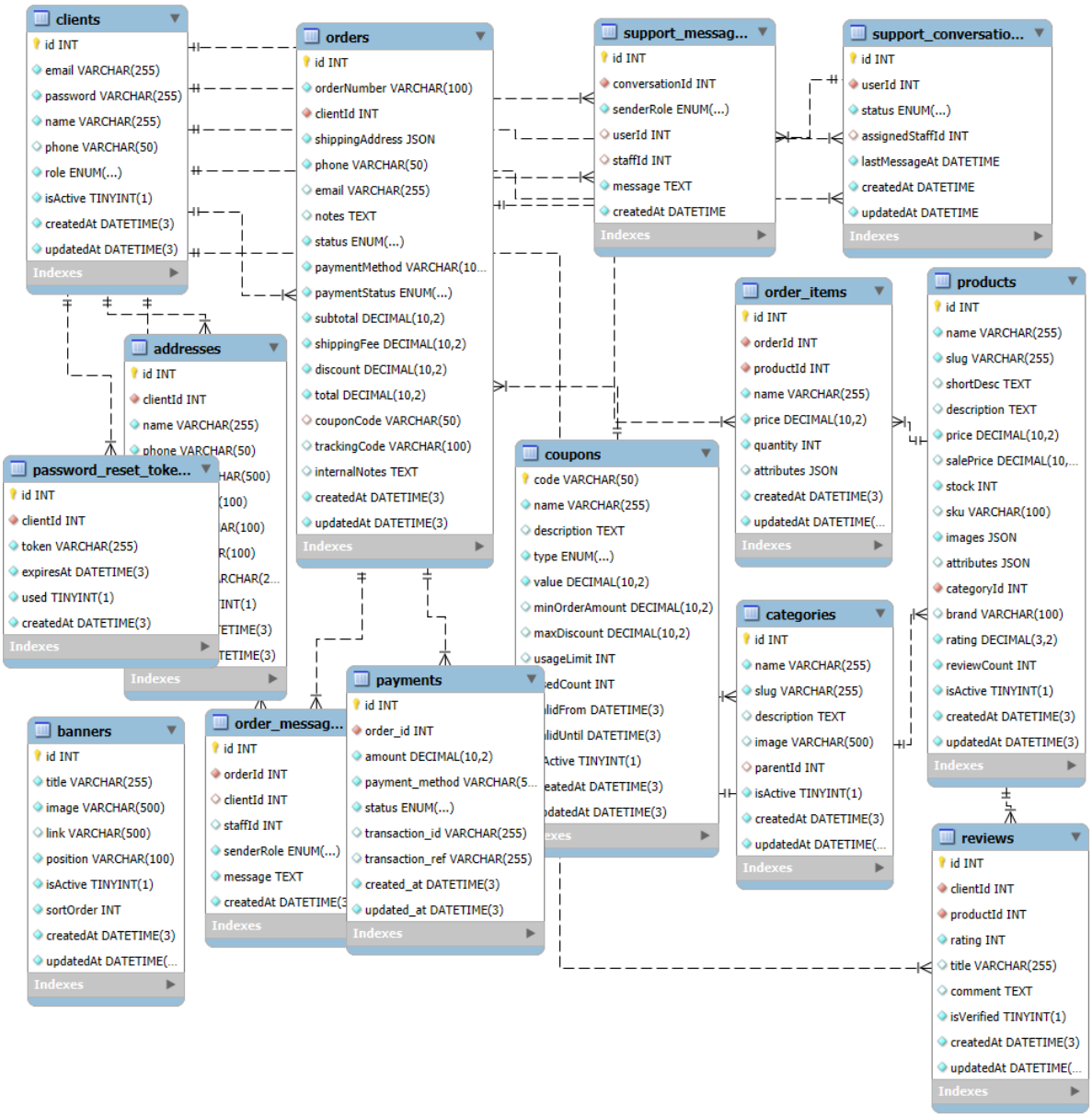

Figure 1.ERD

1.1.3. Redis

We used a Redis cloud database for store refresh token, caching frequently used data,..

1.1.4. Swagger

Our project utilizes Swagger, set of tools built around the OpenAPI Specification, mainly used for designing, documenting, and testing RESTful APIs. It generates interactive documentation that lets users explore and test API endpoints directly in the browser. Swagger is especially useful for creating clear, standardized API docs and for generating client/server code from API definitions.

Figure 2:Backend API Documentation

II.Methodology

2.1 Sequence Diagrams

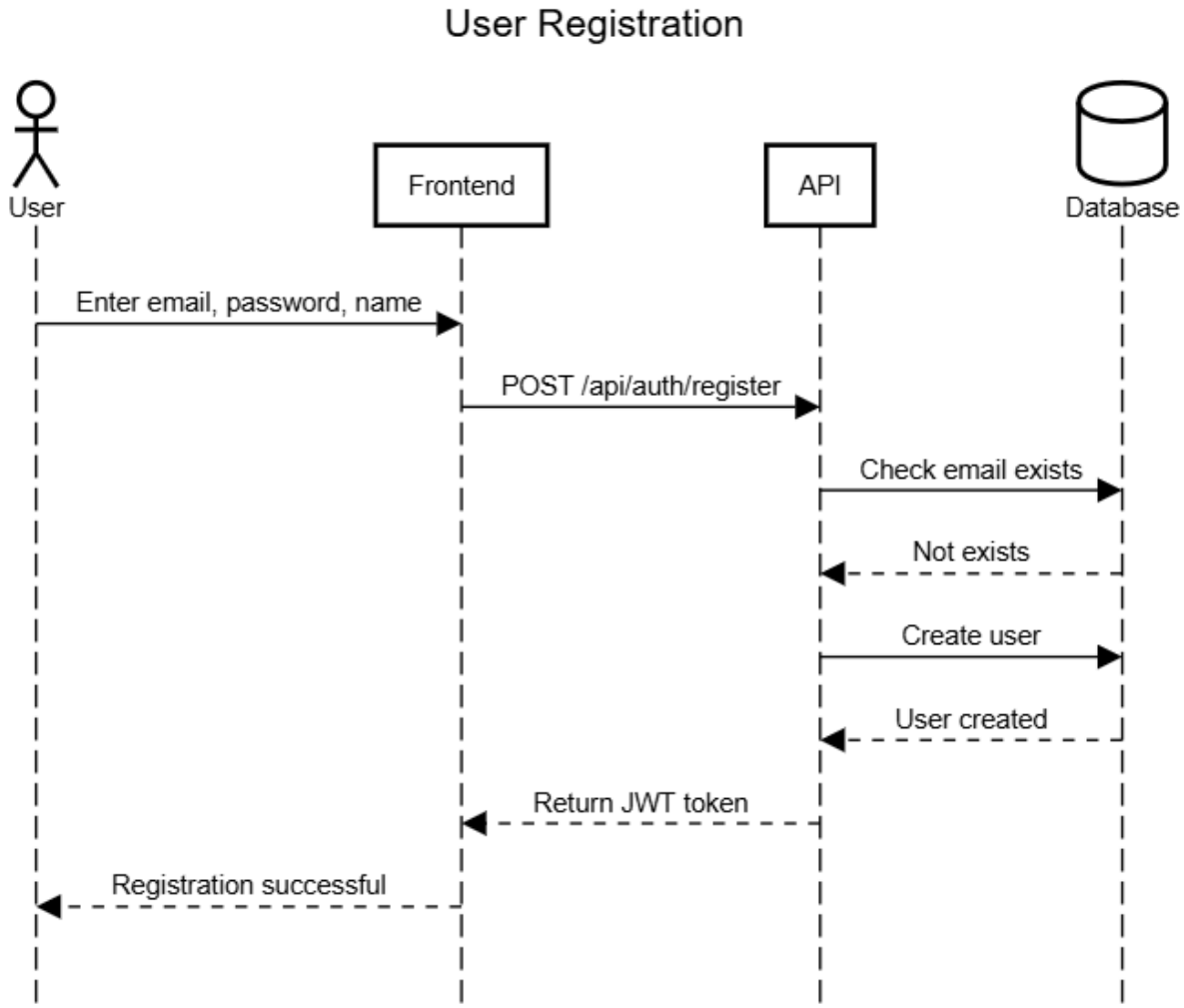

Figure 3:User Registration

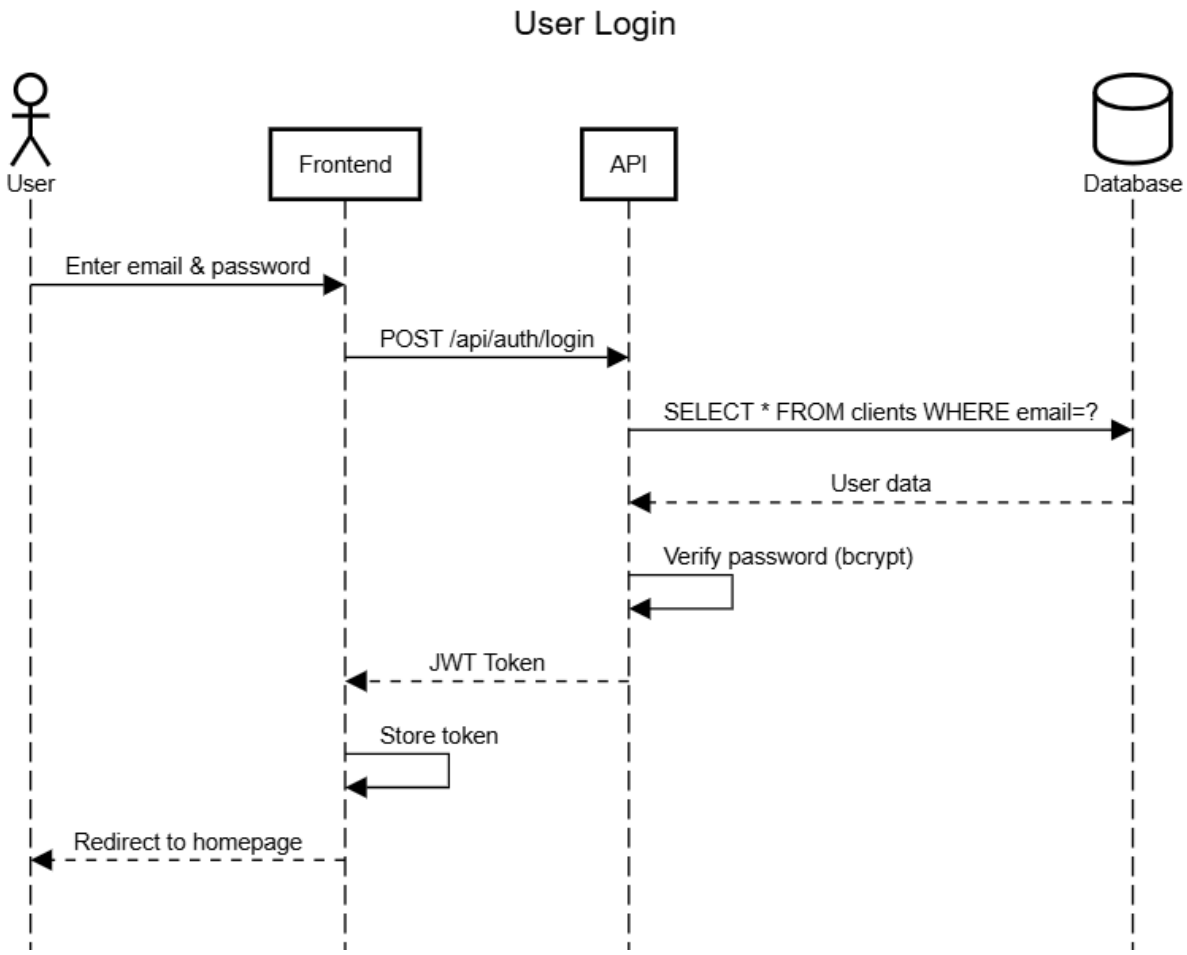

Figure 4:User Login

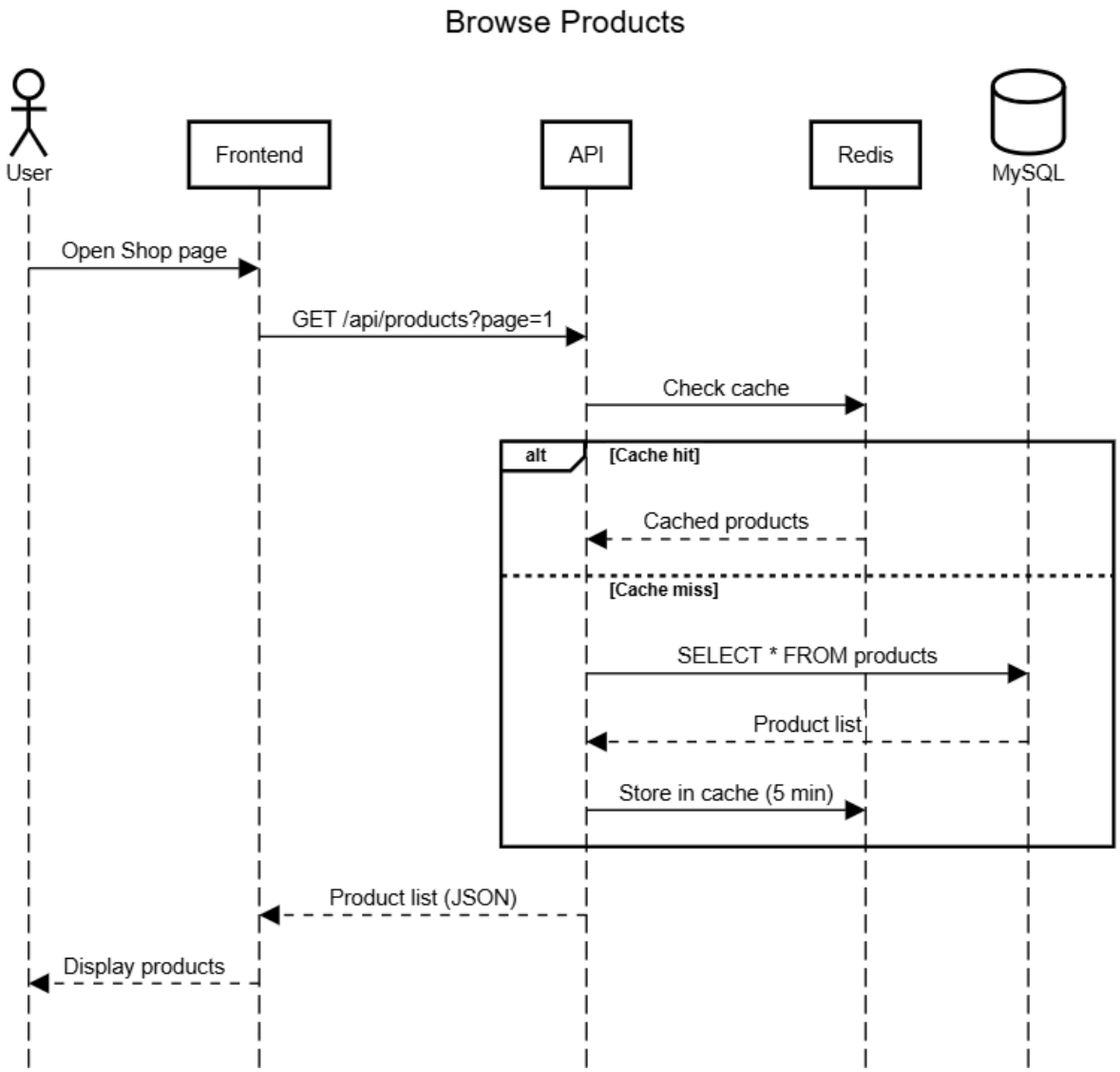

Figure 5:Browse Product

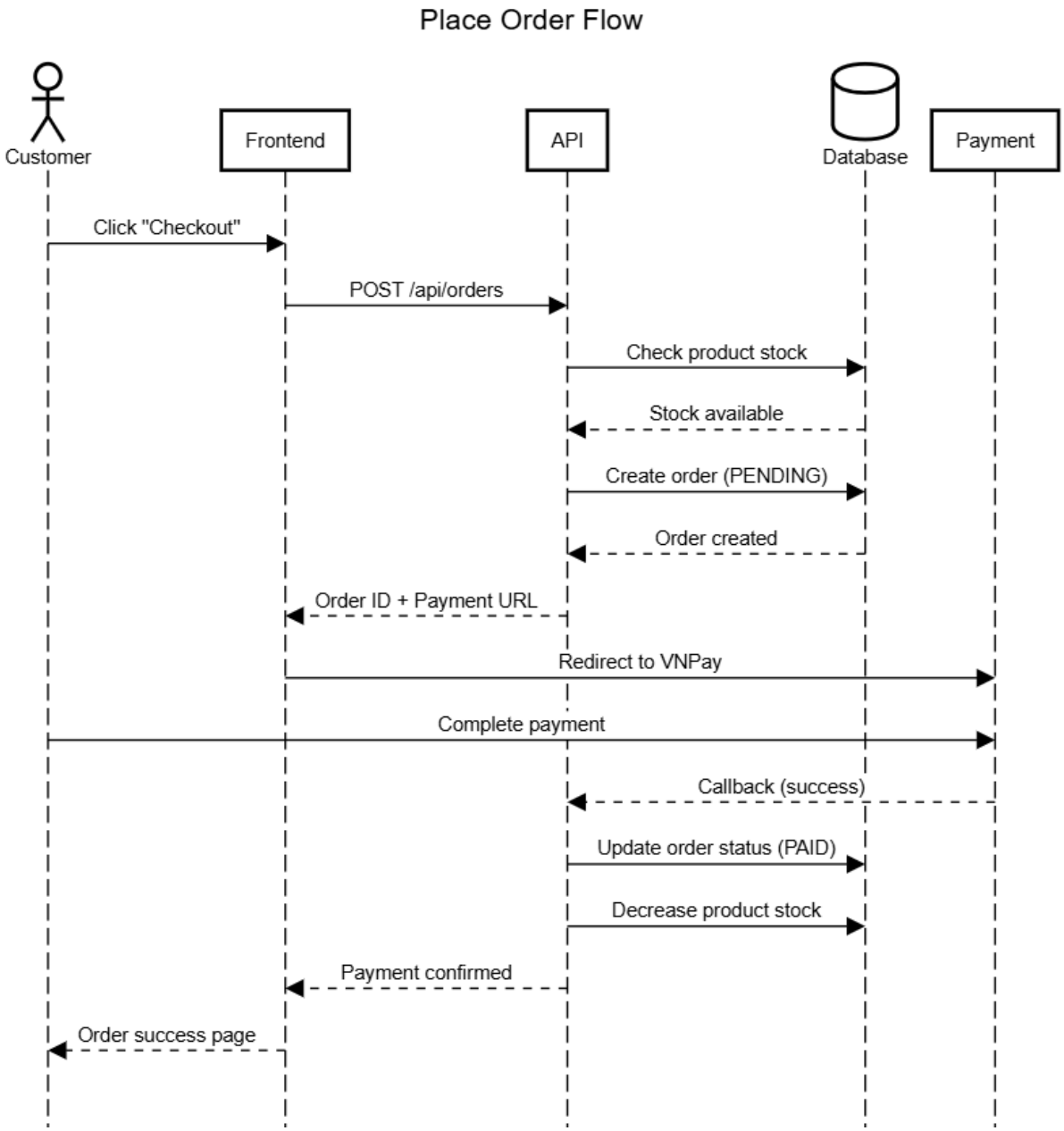

Figure 6:Place Order

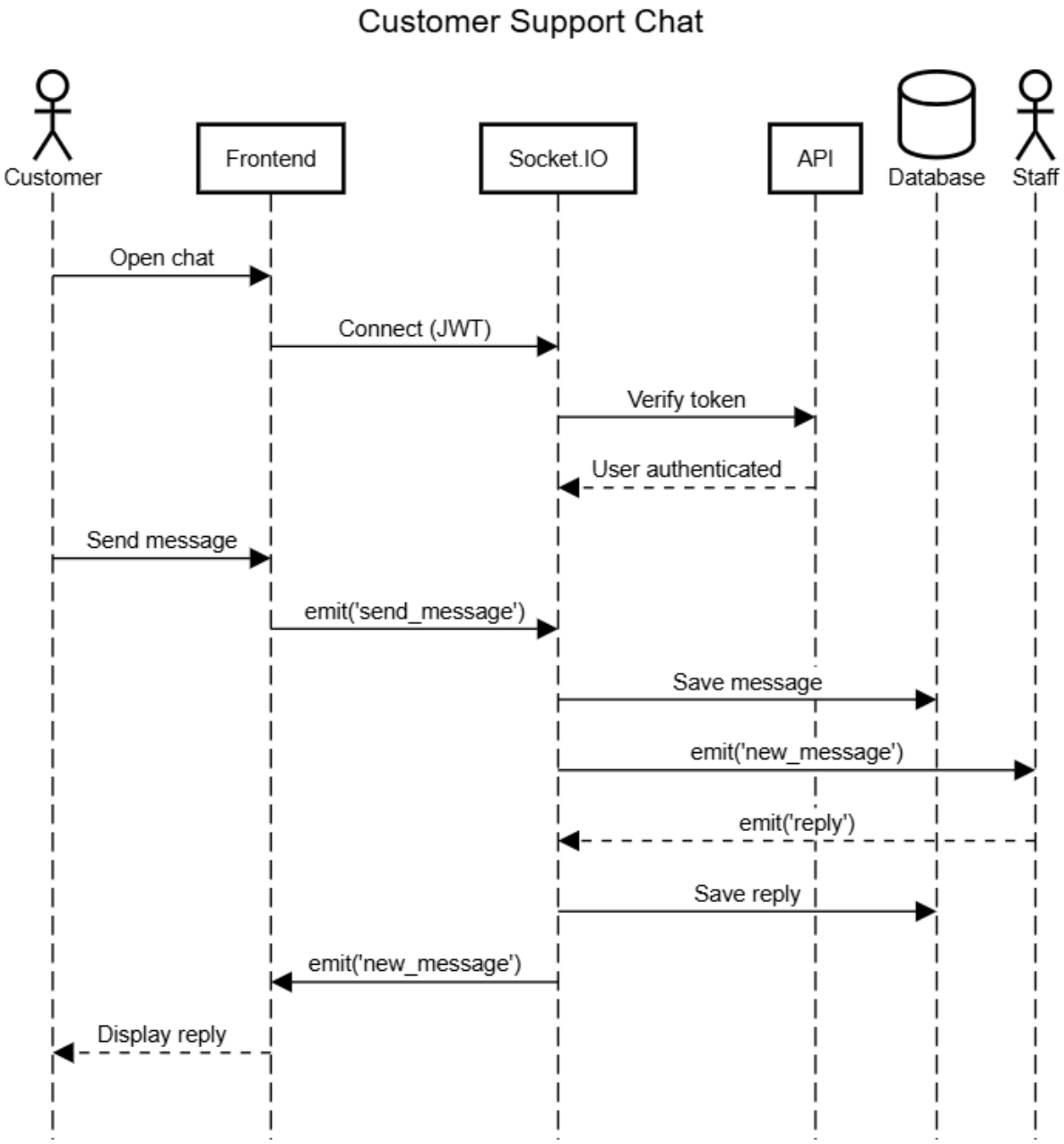

Figure 7:Real-time chat

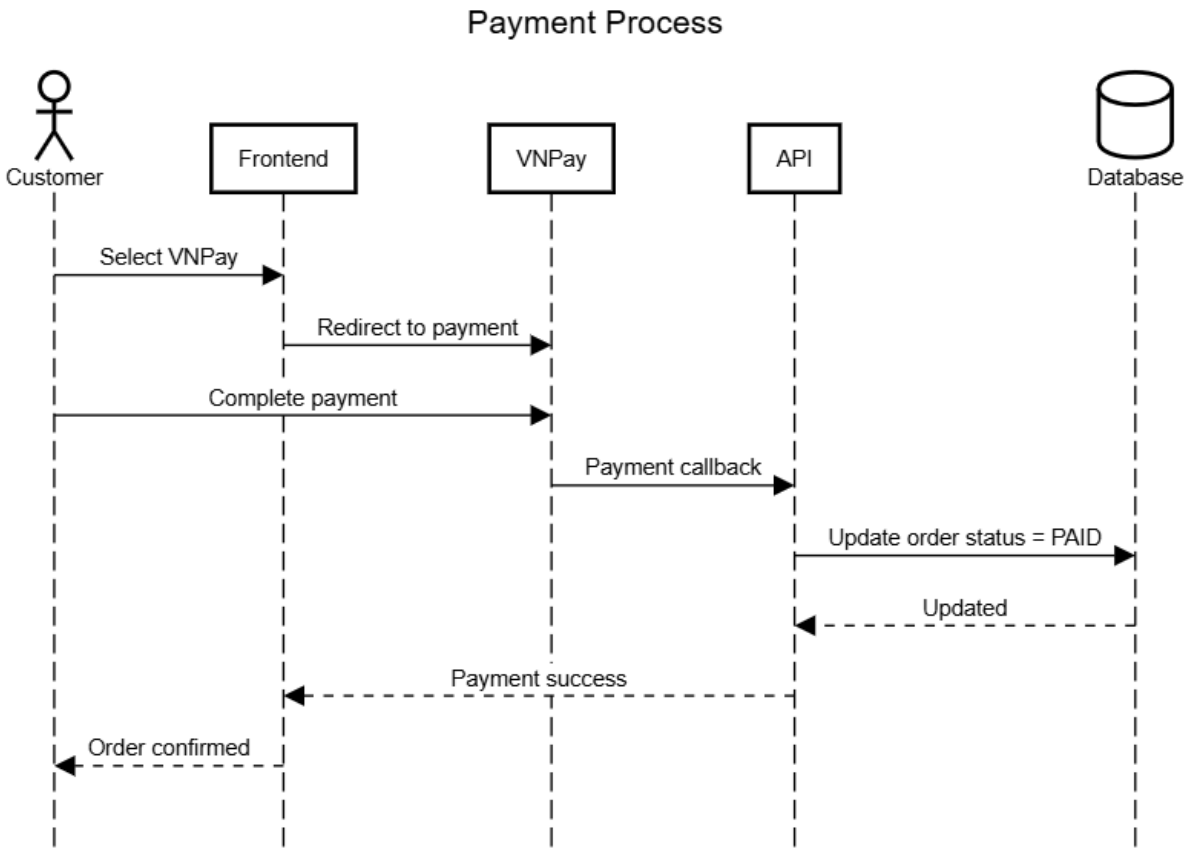

Figure 8:Payment Process

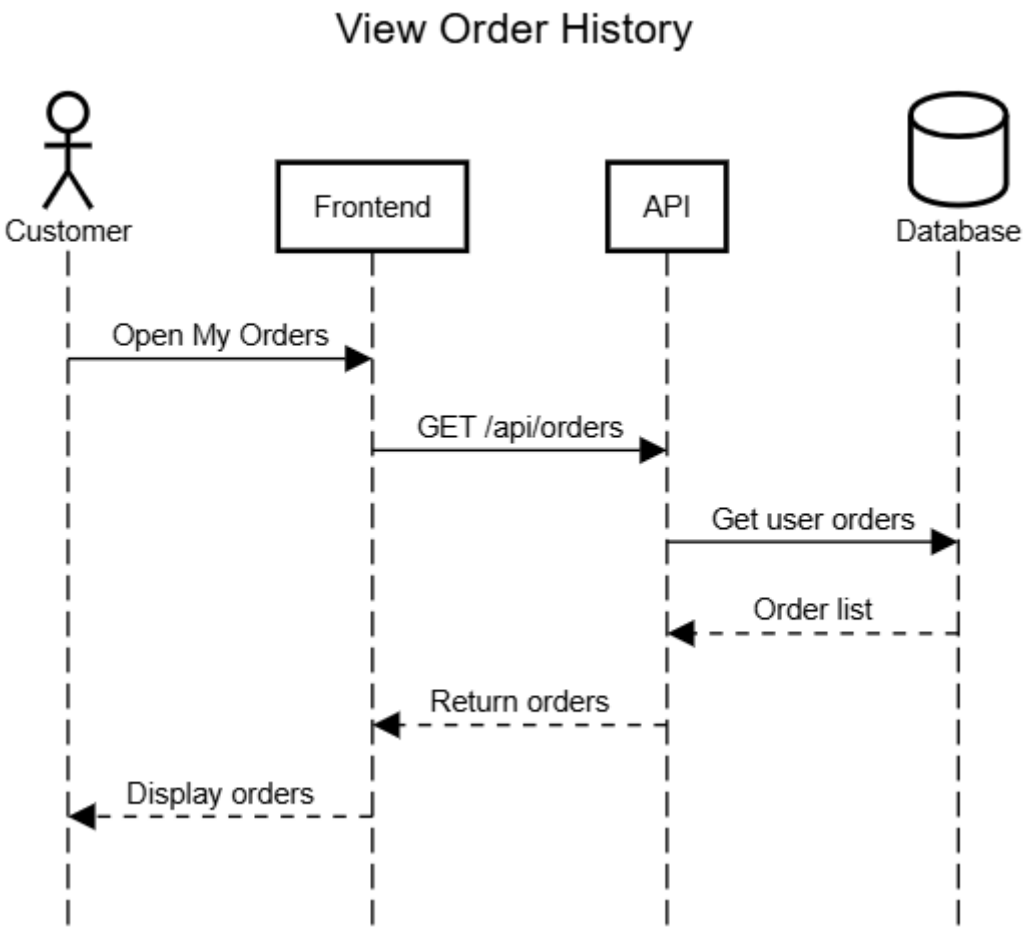

Figure 9:View Order

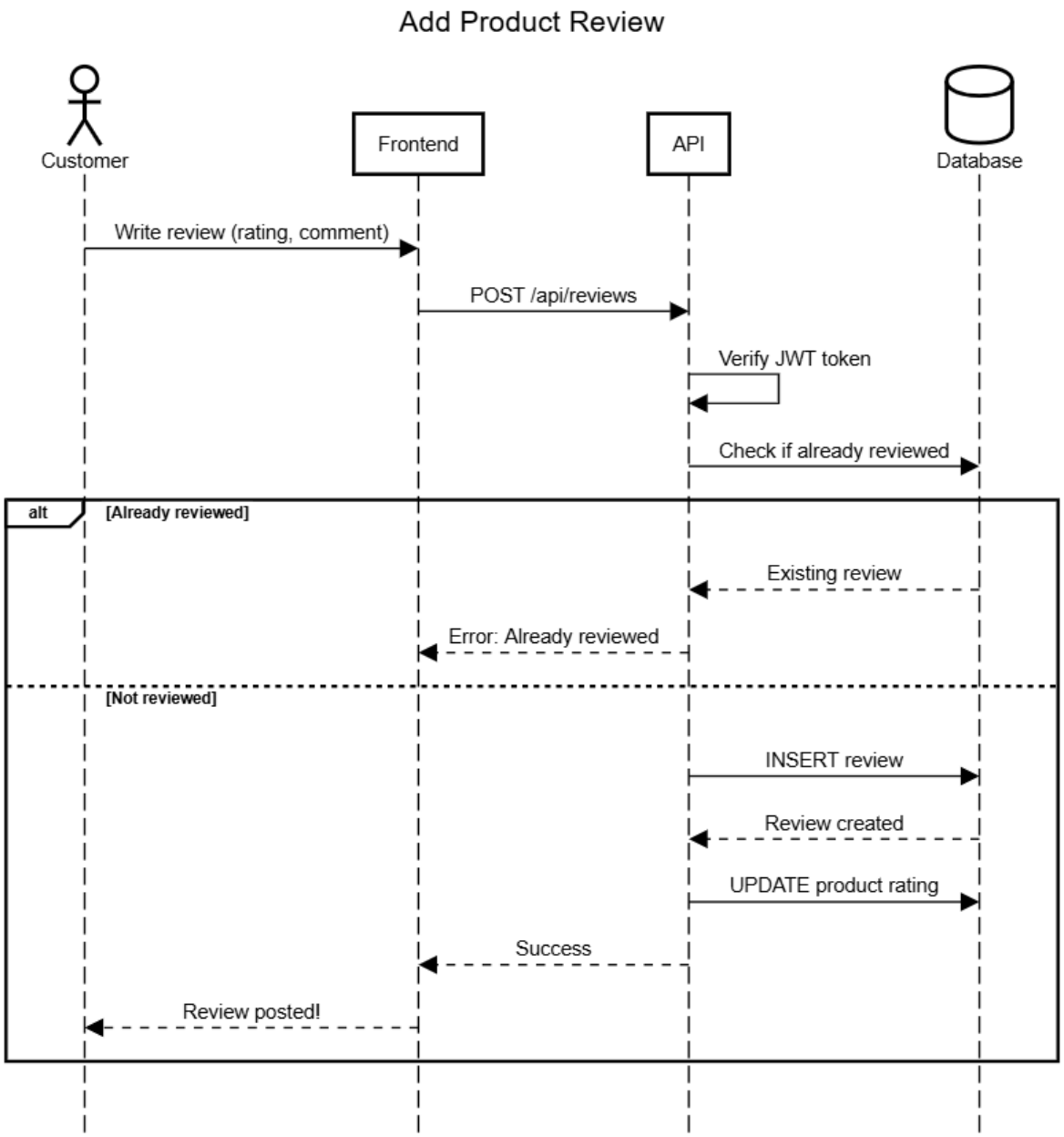

Figure 10:Review

2.2. Backend Implementation Overview

The backend of the system is implemented using the Model–View–Controller (MVC) architecture to ensure clear separation of concerns, scalability, and maintainability. The client communicates with the backend through RESTful APIs. Incoming requests are first handled by Controllers, which validate input data, apply business rules, and coordinate the processing flow. Controllers then interact with Models to perform data operations, where Models represent the database entities and handle

queries/transactions with the MySQL database. Finally, the backend returns structured JSON responses to the client.

Overall, the backend follows an MVC request flow: Client → Routes → Controllers → Models → Database → JSON Response, as illustrated in Figure

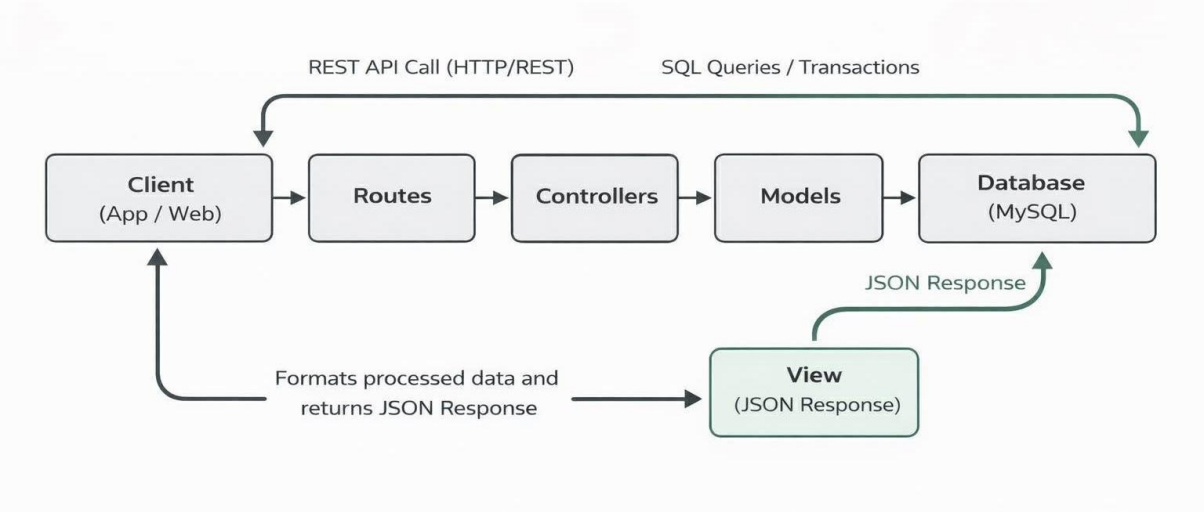

Figure 11:Backend flow

2.2.1 Project Structure

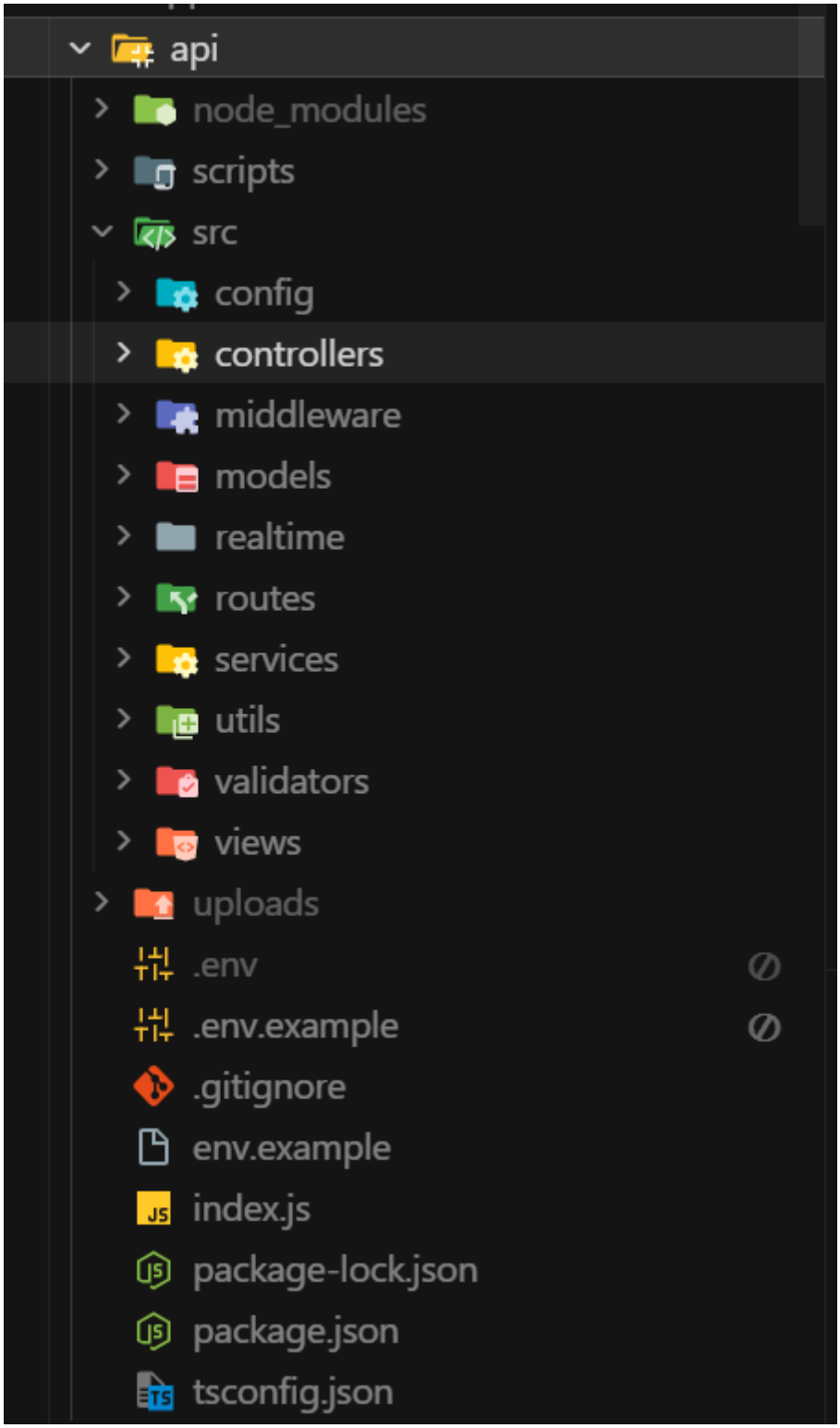

Figure 12:Backend project folder structure

Main folders of the projects are:

+ Routing Layer :The routing layer defines all RESTful API endpoints of the system. Each router groups endpoints by functional domain (e.g.,

authentication, products, orders) and maps HTTP requests to controller methods.

Example:

```
import express from 'express';
import * as orderController from '../../controllers/admin/order.controller.js';
import { authenticate, authorize } from '../../middleware/auth.js';

const router = express.Router();

router.use(authenticate, authorize('ADMIN', 'STAFF'));
router.get('/', orderController.getOrders);
router.get('/:id', orderController.getOrder);
router.put('/:id/status', orderController.updateOrderStatus);
router.get('/:id/payment', orderController.getOrderPayment);
router.put('/:id/payment-status', orderController.updateOrderPaymentStatus);

export default router;
```

Figure 13:Order router mapping admin API endpoints to controllers.

+ Controller Layer : Controllers handle incoming HTTP requests from routers. They extract request data, perform validation, invoke business logic, and return standardized responses to the client with appropriate HTTP status codes.
Example:

```
export const createOrder = async (req, res, next) => {
  try {
    // Validate request body
    const validationResult = createOrderSchema.safeParse({ body: req.body });

    if (!validationResult.success) {
      const errors = validationResult.error.errors.map(err => ({
        field: err.path.join('.'),
        message: err.message,
      }));
      return res.status(400).json({
        error: 'Validation error',
        details: errors,
        message: errors[0]?.message || 'Invalid request data'
      });
    }
```

Figure 14:Create Order controller

+ Service Layer: The service layer contains the core business logic of the application. It enforces domain rules and coordinates complex workflows such as order processing, price calculation, and discount application.

Example:

```
export class PaymentService {
  static async updatePaymentStatus(
    orderId: number,
    status: 'PENDING' | 'PAID' | 'FAILED' | 'REFUNDED',
    transactionId?: string
  ): Promise<void> {
    const connection = await beginTransaction();
    try {
      // Convert undefined to null for SQL
      const transactionIdValue = transactionId ?? null;

      // Check if payment record exists
      const [existingPayments] = await connection.execute(
        `SELECT id FROM payments WHERE order_id = ?`,
        [orderId]
      );

      if (Array.isArray(existingPayments) && existingPayments.length === 0) {
        // Payment record doesn't exist, create it
        // Get order info to create payment
        const [orders] = await connection.execute(
          `SELECT total, paymentMethod FROM orders WHERE id = ?`,
          [orderId]
        );
```

Figure 15:. Payment Service

+ Model Layer: The model layer is responsible for interacting with the MySQL database. It encapsulates SQL queries and manages database transactions to ensure data consistency and integrity.

+ Example:

```
async create(orderData) {
  const connection = await beginTransaction();
  try {
    const {
      userId,
      items,
      shippingAddress,
      phone,
      email,
      notes,
      paymentMethod,
      couponCode,
      shippingFee = 0,
    } = orderData;

    let subtotal = 0;
    let discount = 0;

    // Get products
    const productIds = items.map((item) => item.productId);
    const placeholders = productIds.map(() => '?').join(',');
    const [products] = await connection.execute(
      `SELECT * FROM products WHERE id IN (${placeholders}) AND isActive = 1`,
      productIds
    );

    if (products.length !== productIds.length) {
      throw new Error('Some products not found or inactive');
    }
```

Figure 16:. Create Order Model

+ Middleware Layer: Middleware components handle cross-cutting concerns that are applied before requests reach the controller layer. This includes authentication, authorization, and other request preprocessing tasks. By centralizing these concerns, middleware ensures consistent security and request handling across all API endpoints.

Example:

```
import { verifyToken } from '../utils/jwt.js';
import { authClientModel } from '../models/client/auth.model.js';

export const authenticate = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.replace('Bearer ', '');

    if (!token) {
      return res.status(401).json({ error: 'Authentication required' });
    }

    const decoded = verifyToken(token);
    const user = await authClientModel.findById(decoded.userId);

    if (!user) {
      return res.status(401).json({ error: 'Invalid or inactive user' });
    }

    req.user = {
      id: user.id,
      email: user.email,
      name: user.name,
      role: user.role,
      isActive: true,
    };
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Invalid token' });
  }
};
```

Figure 17: Authentication and authorization middleware for securing API endpoints

+ Validation Layer: The validation layer ensures that incoming request data conforms to predefined rules and constraints before being processed by controllers or business logic. This helps prevent invalid or malicious data from entering the system and improves overall reliability.

Example:

```
export const registerSchema = z.object({
  body: z.object({
    email: z.string().email('Invalid email address'),
    password: z.string().min(6, 'Password must be at least 6 characters'),
    name: z.string().min(2, 'Name must be at least 2 characters'),
    phone: z.string().optional(),
  }),
});

export const loginSchema = z.object({
  body: z.object({
    email: z.string().email('Invalid email address'),
    password: z.string().min(1, 'Password is required'),
  }),
});
```

Figure 18:Authentication request validation schema

+ Utility Layer: The utility layer contains reusable helper functions that support multiple backend modules. These utilities handle common tasks such as authentication helpers, password hashing, logging, caching, and data formatting. Centralizing these helpers helps reduce code duplication and improves maintainability.

Example:

```
const JWT_REFRESH_EXPIRES_IN = config.jwt.refreshExpiresIn;

/**
 * Generate a JWT access token for a user
 * @param {number} userId - User ID to include in token
 * @returns {string} JWT token string
 * @throws {Error} If token generation fails
 */
export const generateToken = (userId: number): string => {
  return jwt.sign({ userId }, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN } as SignOptions);
};

/**
 * Generate a JWT refresh token for a user
 * @param {number} userId - User ID to include in token
 * @returns {string} JWT refresh token string
 * @throws {Error} If token generation fails
 */
export const generateRefreshToken = (userId: number): string => {
  return jwt.sign({ userId, type: 'refresh' }, JWT_SECRET, { expiresIn: JWT_REFRESH_EXPIRES_IN } as SignOptions);
};

/**
 * Verify and decode a JWT token
 * @param {string} token - JWT token string to verify
 * @returns {{ userId: number; exp?: number; iat?: number; type?: string }} Decoded token payload
 * @throws {Error} If token is invalid or expired
 */
export const verifyToken = (token: string): { userId: number; exp?: number; iat?: number; type?: string } => {
  return jwt.verify(token, JWT_SECRET) as { userId: number; exp?: number; iat?: number; type?: string };
};
```

Figure 19: JWT utility functions for token generation and verification

+ Realtime Layer: The realtime layer enables bidirectional communication between the server and connected clients using WebSocket or Socket.IO. This layer supports real-time features such as order status updates, notifications, or customer support chat without relying on traditional HTTP request-response cycles.
  Example:

```
export function initSocket(server) {
  const io = new Server(server, {
    cors: {
      origin: config.corsOrigin,
      methods: ['GET', 'POST'],
      credentials: true,
    },
  });

  io.use((socket, next) => {
    const token = socket.handshake.auth?.token || socket.handshake.query?.token;

    if (!token || typeof token !== 'string') {
      return next(new Error('Authentication error'));
    }

    try {
      const decoded = verifyToken(token.replace('Bearer ', ''));
      socket.data.userId = decoded.userId;
      socket.data.role = decoded.role;
      next();
    } catch (error) {
      next(new Error('Authentication error'));
    }
  });
```

Figure 20:Realtime module handling WebSocket-based communication

2.3. Front-end Overview and Architecture

The front-end follows a layered architecture consisting of UI Components, State Management, and an API Service Layer. User interactions are handled by UI components, which update the centralized state using Zustand. The API service layer communicates with the backend via RESTful APIs. Real-time features are supported through a Socket.IO server using WebSocket connections, enabling live updates without page reloads.

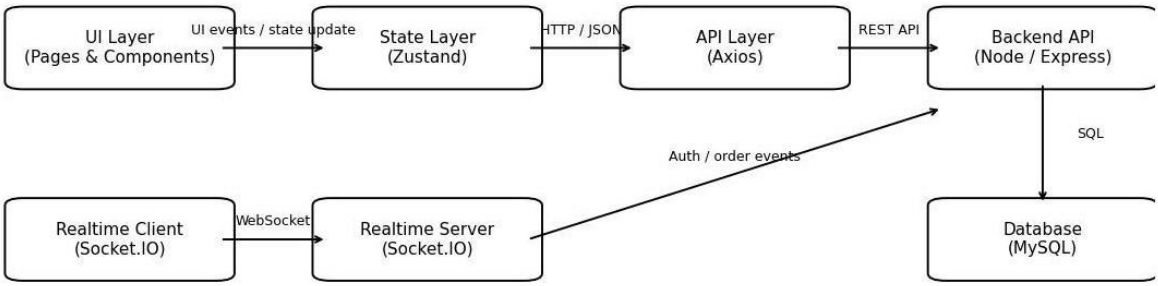

Figure 21: Front-end architecture overview

.

2.3.1 Project Structure

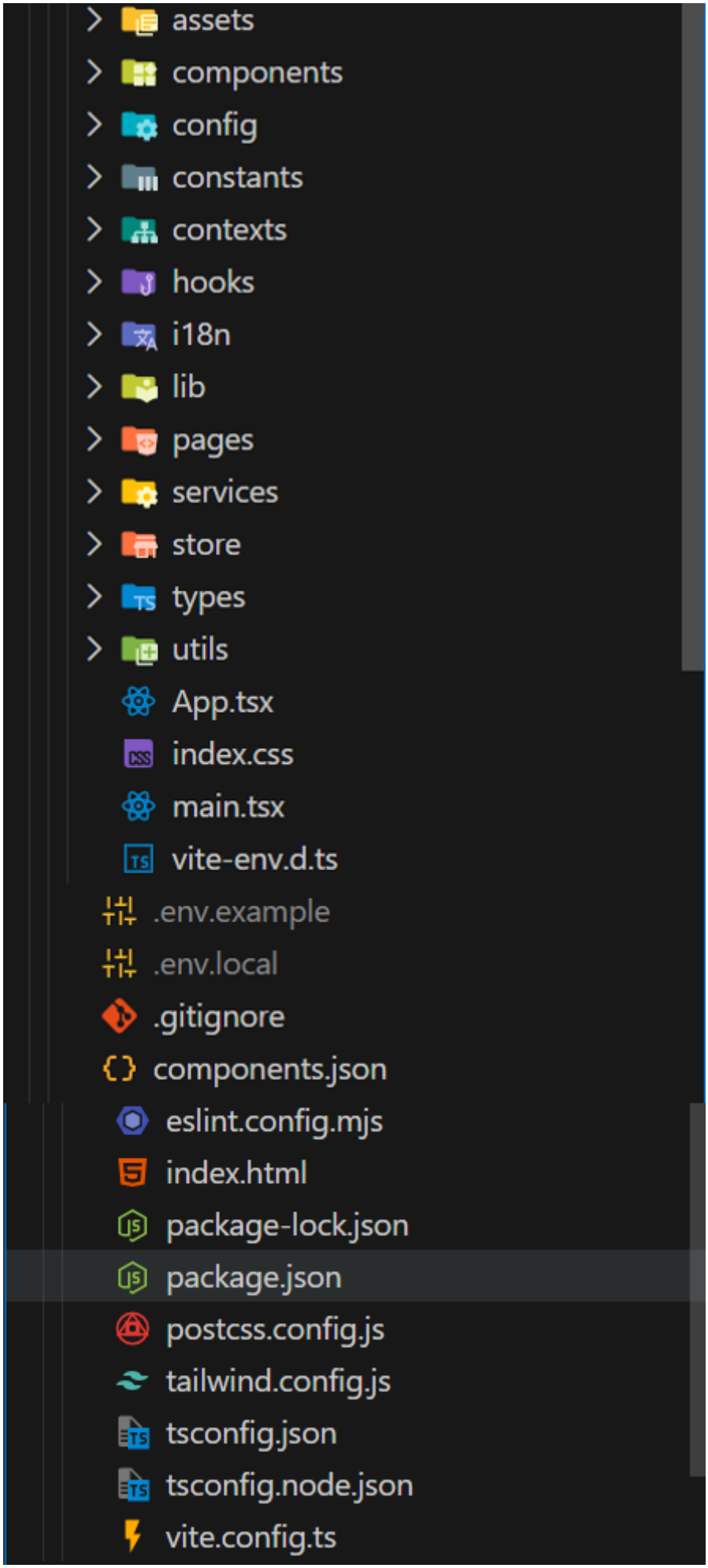
assets
components
config
constants
contexts
hooks
i18n
lib
pages
services
store
types
utils
App.tsx
index.css
main.tsx
vite-env.d.ts
.env.example
.env.local
.gitignore
components.json
eslint.config.mjs
index.html
package-lock.json
package.json
postcss.config.js
tailwind.config.js
tsconfig.json
tsconfig.node.json
vite.config.ts

Figure 22: Front-end project folder structure.

Each module and logic is separated clearly to ensure modular and clean code with minimal dependency. The clean separation between API calls, views, components and theme makes it easier for us to ensure maintainability and consistency throughout the system, with high level of code resuablity.

+ assets: This folder contains static resources such as images, icons, and fonts used throughout the user interface.

+ components: This folder includes reusable UI components such as buttons, cards, forms, and navigation elements that are shared across multiple pages.

+ config: This folder stores front-end configuration files, such as environment-specific settings or application constants.

+ constants: This folder defines constant values (e.g., enums, fixed strings, default values) used across the front-end to ensure consistency and reduce duplication.

+ contexts: This folder contains React Context definitions used for global state management, such as authentication state and notification handling, allowing data to be shared across components without excessive prop drilling.

+ hooks: This folder includes custom React hooks that encapsulate reusable logic, improving code readability and reusability.

+ i18n: This folder manages internationalization configuration and translation resources to support multiple languages.

+ lib: This folder provides shared libraries or wrapper utilities for third-party integrations used by the front-end.

+ pages: This folder contains page-level components that represent the main views of the application, such as product listing, product details, cart, and checkout pages.

+ services: This folder handles communication with the backend API, centralizing all HTTP requests and responses.

+ store: This folder manages global application state using a state management solution (e.g., Redux or equivalent), enabling consistent data sharing across the application.

+ types: This folder defines TypeScript type declarations and interfaces to ensure type safety across the front-end codebase.

+ utils: This folder contains utility helper functions shared across different front-end module

2.3.2 API Calls

In the e-commerce application, the front-end communicates with the backend using RESTful API calls implemented with Axios. A centralized Axios instance is used to manage HTTP requests and automatically attach authentication tokens through request interceptors. This approach ensures consistent and secure communication between the front-end and backend services.

```
const api = axios.create({
  baseURL: config.api.baseURL,
  timeout: config.api.timeout,
  headers: {
    'Content-Type': 'application/json',
  },
});


api.interceptors.request.use(
  (config) => {

    if (
      config.url?.includes('/auth/login') ||
      config.url?.includes('/auth/register')
    ) {
      delete config.headers.Authorization;
      return config;
    }

    const token = localStorage.getItem(STORAGE_KEYS.TOKEN);
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    } else {
      delete config.headers.Authorization;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
```

Figure 23: Example of API call with Axios.

2.3.2. State Management

Global application state, including user authentication and shopping cart data, is managed centrally on the front-end. This allows shared data to be accessed

consistently across multiple pages and components, which is essential for core e-commerce features such as cart updates and user sessions.

```
export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      isAuthenticated: false,

      login: async (email: string, password: string) => {
        // Clear any existing auth data before login
        localStorage.removeItem('token');
        localStorage.removeItem('auth-storage');

        const response = await api.post('/auth/login', { email, password });
        const { user, token } = response.data;

        // Set new token and state
        localStorage.setItem('token', token);
        set({ user, token, isAuthenticated: true });
      },

      register: async (email: string, password: string, name: string, phone?: string) => {
        const response = await api.post('/auth/register', { email, password, name, phone });
        const { user, token } = response.data;
        localStorage.setItem('token', token);
        set({ user, token, isAuthenticated: true });
      },
```

Figure 24:Global authentication state management on the front-end.

2.3.3. Pages and Components

The front-end user interface is built using a combination of page-level components and reusable UI components. Pages represent the main views of the e-commerce application, such as product listing, product details, cart, and checkout, while reusable components are shared across multiple pages to ensure UI consistency. This component-based approach improves code reusability and simplifies maintenance of the user interface.

```tsx
export default function ProductDetailPage() {
  const { slug } = useParams();
  const navigate = useNavigate();
  const [product, setProduct] = useState<Product | null>(null);
  const [reviews, setReviews] = useState<Review[]>([]);
  const [quantity, setQuantity] = useState(1);
  const [loading, setLoading] = useState(true);
  const [activeImage, setActiveImage] = useState(0);
  const [activeTab, setActiveTab] = useState<'description' | 'reviews'>('description');
  const [showReviewForm, setShowReviewForm] = useState(false);
  const [reviewRating, setReviewRating] = useState(5);
  const [reviewTitle, setReviewTitle] = useState('');
  const [reviewComment, setReviewComment] = useState('');
  const [submittingReview, setSubmittingReview] = useState(false);
  const { addItem } = useCartStore();
  const { isAuthenticated } = useAuthStore();
  const { toast } = useToast();

  useEffect(() => {
    if (slug) {
      fetchProduct();
    }
  }, [slug]);

  const fetchProduct = async () => {
    try {
      const res = await api.get(`/products/slug/${slug}`);
      setProduct(res.data);
      if (res.data.reviews) {
        setReviews(res.data.reviews);
      }
```

Figure 25: Product detail page

2.3.4 Features using extra libraries

To enhance the user experience and development efficiency of the e-commerce application, several features were implemented using third-party npm libraries:

- HTTP Client: https://www.npmjs.com/package/axios
- State Management: https://www.npmjs.com/package/zustand
- Routing: https://www.npmjs.com/package/react-router-dom
- Form Management: https://formik.org/
- Animation: https://www.npmjs.com/package/framer-motion
- Styling: https://tailwindcss.com/

In conclusion, the front-end of the e-commerce system was developed with a strong focus on modularity, maintainability, and user experience. The use of modern libraries

and frameworks enables efficient development while ensuring seamless integration with backend services and a responsive shopping experience.

2.5. Database Overview

The system uses a relational database (MySQL) to store and manage all data required for the Book Store E-Commerce Website. The database is designed to support core functionalities such as user management, book catalog browsing, shopping cart operations, and order processing. A normalized relational structure with clearly defined primary keys and foreign keys is applied to ensure data integrity, consistency, and scalability. The database design also allows future extensions, such as vouchers, reviews, or payment integration, without major structural changes.

2.5.1. Core Entities

The main entities in the database include:

+ Users: stores user account information, authentication data, and user roles.
+ User_Addresses: manages multiple shipping addresses for each user.
+ Categories: organizes books into categories and supports hierarchical relationships.
+ Books: stores detailed information about books, including price, stock, and category.
+ Carts: represents the active shopping cart of a user.
+ Cart_Items: stores books added to the cart along with quantity and unit price.
+ Orders: records completed purchases made by users.
+ Order_Items: stores detailed information of each book included in an order.

2.5.2 Relationship Description

Each User can have multiple Addresses, Orders, and an active Cart. The Cart and Order entities are linked to Books through the intermediate tables Cart_Items and Order_Items, which store additional attributes such as quantity and price at the time of transaction. Each Book belongs to a specific Category, enabling efficient browsing

and filtering. These relationships ensure accurate data management throughout the shopping and checkout process.

2.5.3 Entity Relationship Diagram

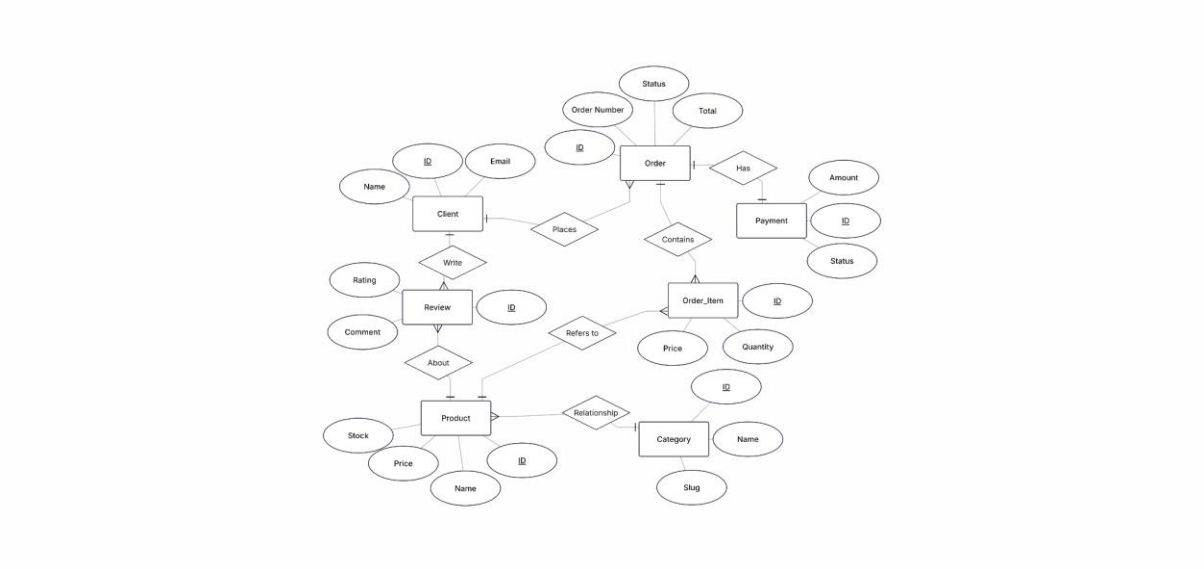

Figure 26: Entity Relationship Diagram (ERD) of the E-Commerce System

III.Demo

1. Authentication

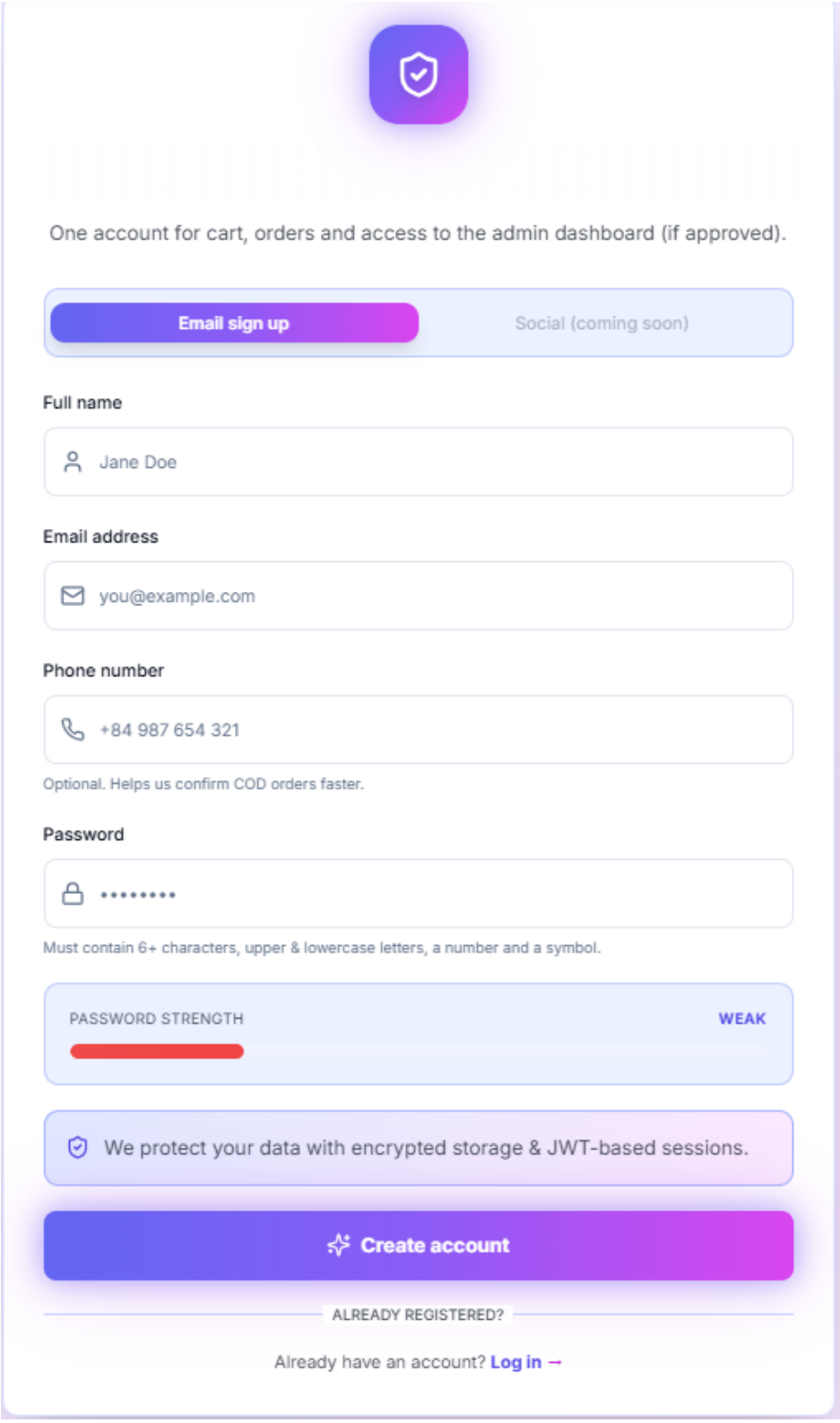

Figure 27:Sign Up Form

First, the user opens the Sign Up page and selects the Email sign up option.

Next, the user enters their Full name, Email address, and an optional Phone number to complete the basic account information.

Then, the user creates a Password that meets the security requirements, including a minimum length and a combination of uppercase letters, lowercase letters, numbers, and symbols. A password strength indicator is displayed in real time.

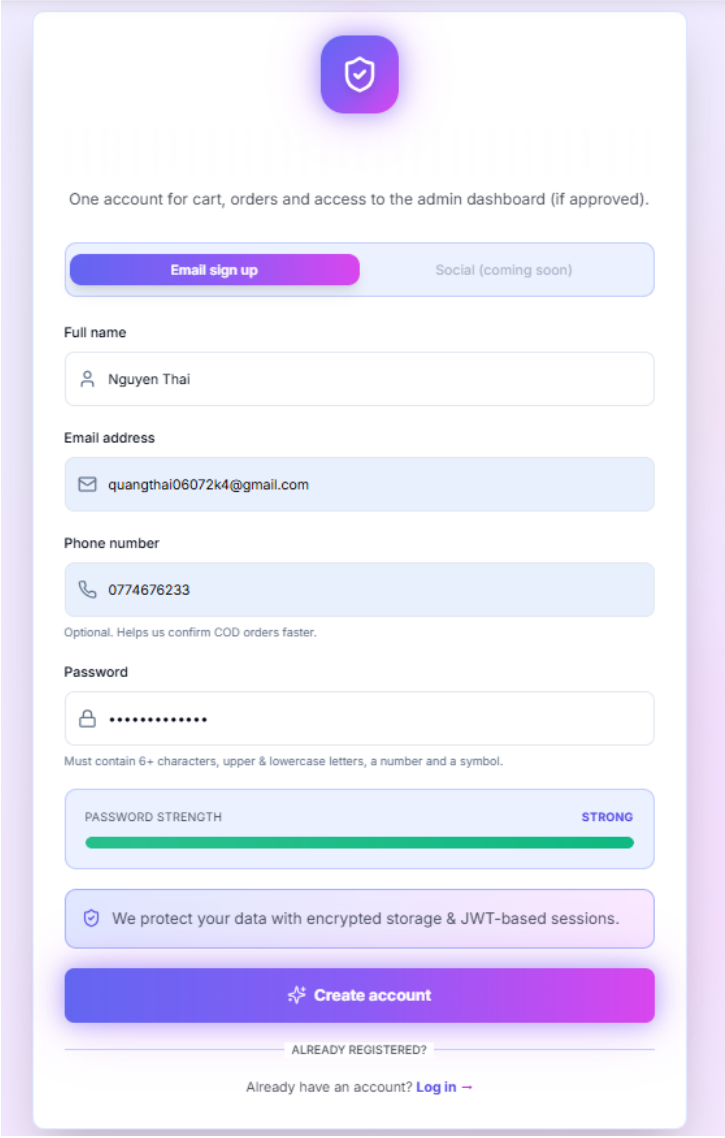

Figure 28: Sign Up Form

After all required fields are validated, the user clicks Create account. The system creates the account and automatically logs the user into the website without requiring a separate login step.

Finally, the user is redirected to the main interface of the e-commerce website and can immediately access features such as browsing products, managing the shopping cart, and placing orders.

2. Product Browsing

2.1 Home Page

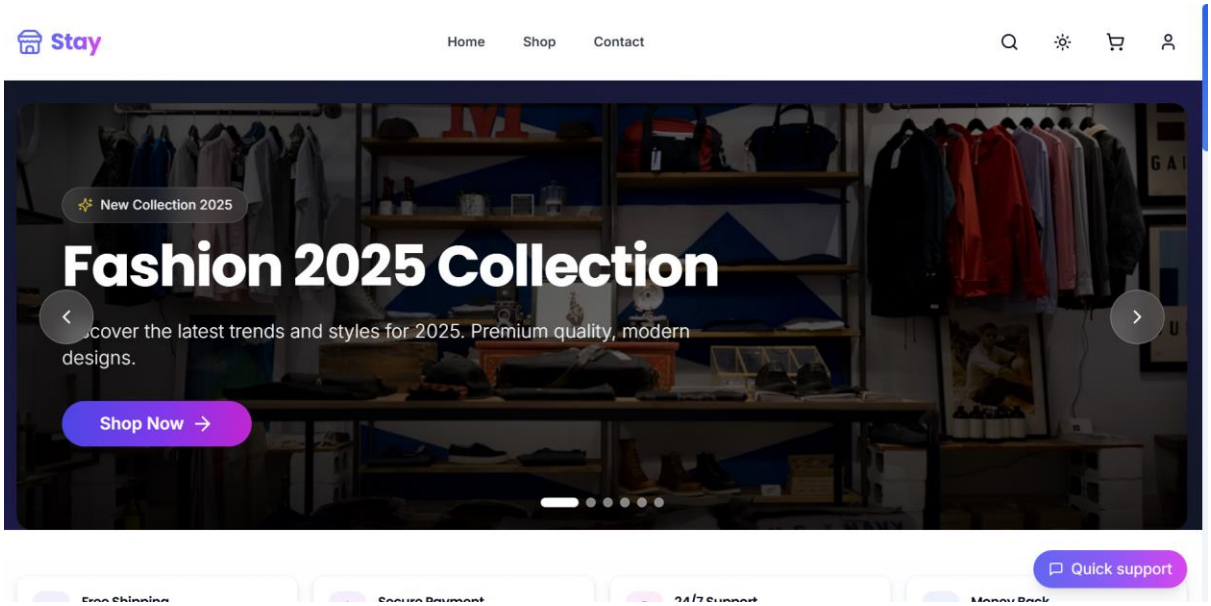

Figure 29: Stay Home Page

After successful login, the user is redirected to the Home Page. The hero section highlights the latest fashion collection with a promotional banner and a Shop Now button, allowing users to quickly access featured products.\

2.2. Search

The header provides main navigation options, including Home, Shop, and Contact, along with user-related actions such as account access, shopping cart, and theme settings.

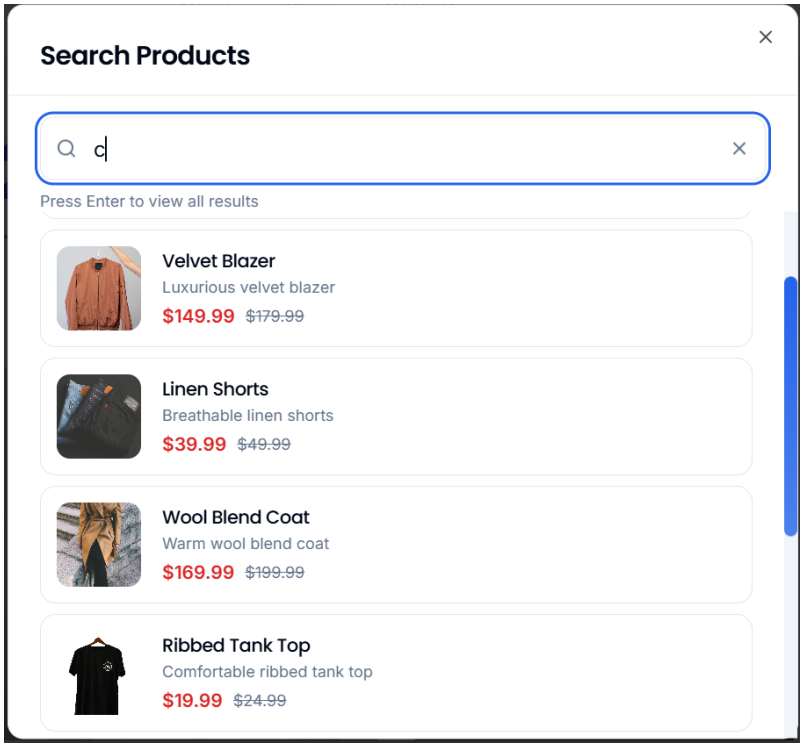

Figure 30: Search Function in Header

The search function is accessible directly from the header through a search icon. When users click the search icon, a search dialog appears, allowing them to enter keywords and quickly find products without leaving the current page.

2.3 Shop

The Shop All Products page displays the complete list of products available in the system, including product images, categories, pricing details, and discount indicators. Users can refine product results using the filter panel on the left, which supports searching by keyword, filtering by category, and selecting a price range. Additionally, products can be sorted by criteria such as Newest Arrivals, and each product card provides a quick Add to Cart option, allowing users to add items to their shopping cart directly from the shop page.

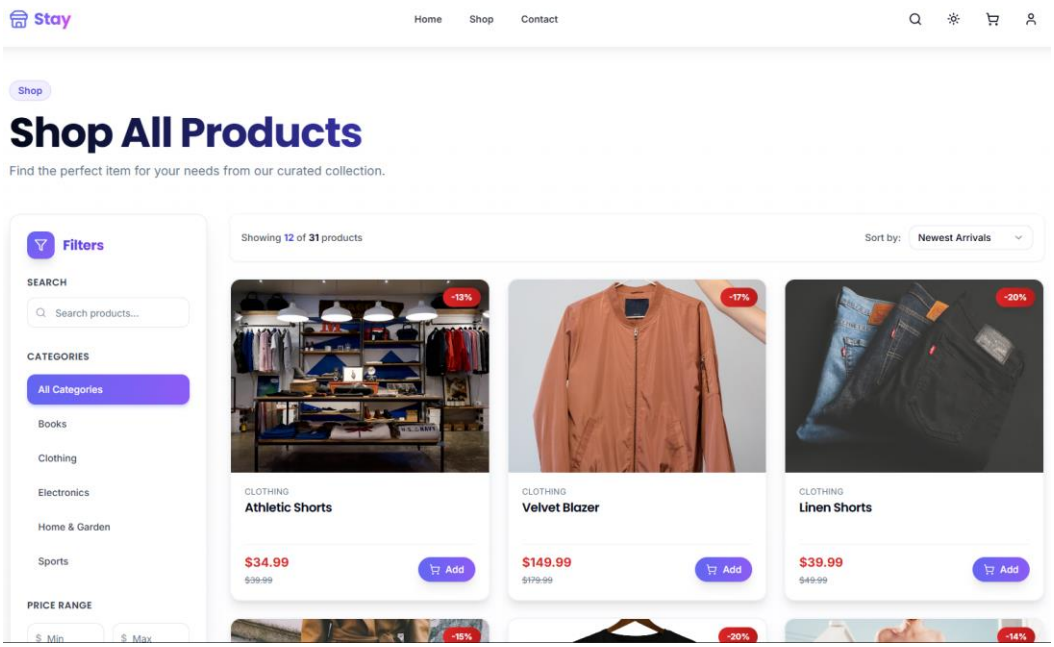

Figure 31: Product Detail Page

After a product is added to the cart, the cart icon in the header is updated with a numeric badge, indicating the current number of items in the shopping cart. This provides immediate feedback to the user.

Figure 32: Add to Cart Action

3. Shopping and Checkout

3.1 Shopping Cart

The Shopping Cart Page displays all products that the user has added to the cart, including product name, price, and selected quantity. Users can increase or decrease the quantity of each item or remove products from the cart. An Order Summary section shows the subtotal, shipping information, and total cost. From this page, users can choose to Proceed to Checkout or Continue Shopping.

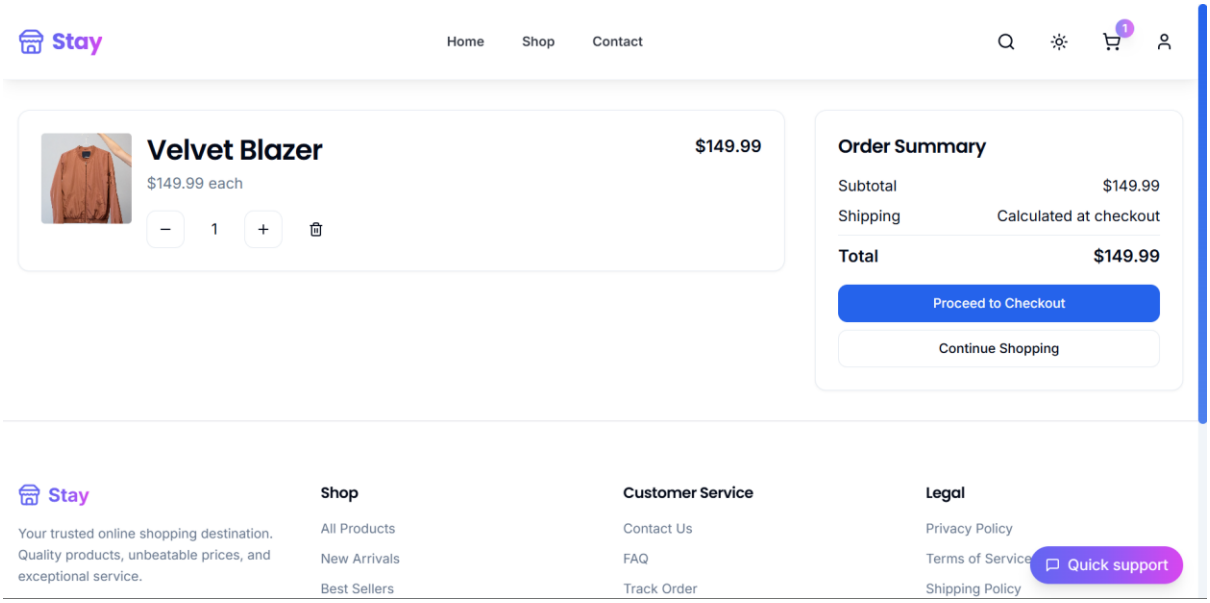

Figure 33: Shopping Cart Page

3.2. Checkout

After adding products to the shopping cart, the user proceeds to the Checkout Page to complete the order. First, the user enters the required shipping address information, including full name, phone number, and delivery address.

Next, the user may enter a coupon code (if available) and select a payment method, such as Cash on Delivery (COD), bank transfer, or e-wallet options. The Order Summary section is updated to reflect the selected items, subtotal, shipping fee, and total amount.

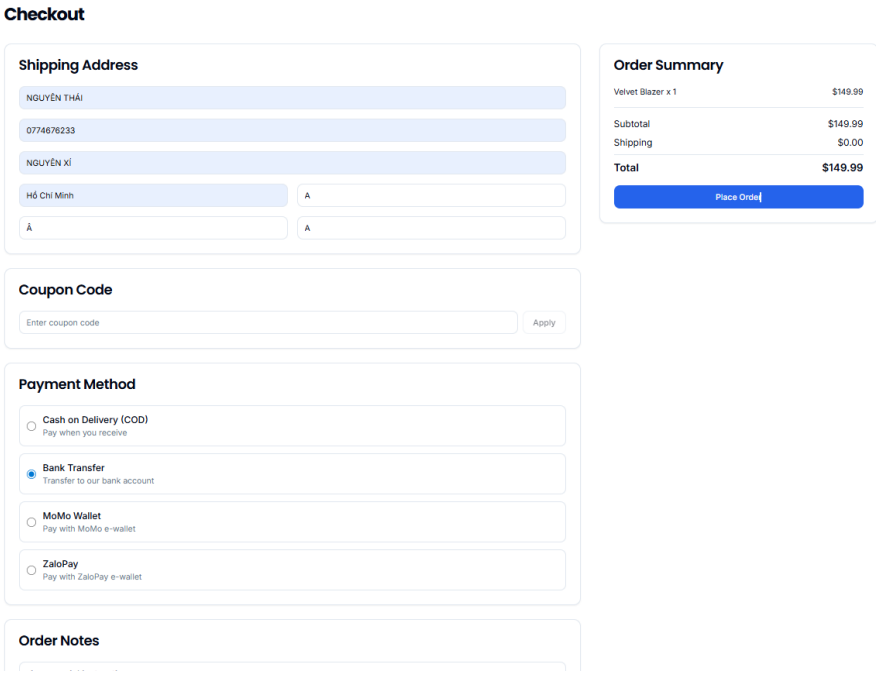

Figure 34: Checkout Page

When the user clicks Place Order, the system simulates the order transaction and automatically marks it as successful. This approach is used to demonstrate the complete checkout workflow in the e-commerce system without integrating a real payment gateway. After the order is placed, the order data is saved in the system, and the checkout process is completed.

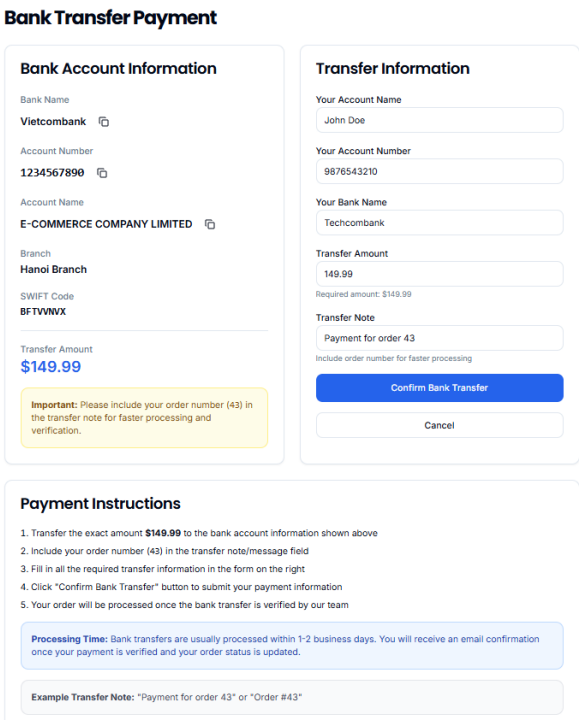

Figure 35: Bank Transfer Payment Page

After entering the required transfer information, the user clicks Confirm Bank Transfer to submit the payment details. The system simulates the bank transfer verification and automatically marks the transaction as successful. Once confirmed, the order status is updated, and the payment process is completed successfully.

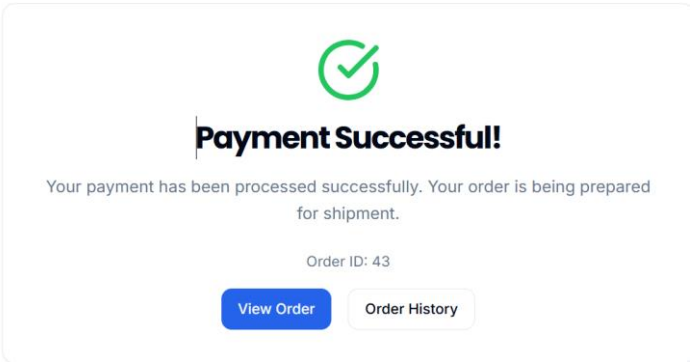

Figure 36:Payment Successful Page

3.3 Order History

After completing the checkout process, users can navigate to the Order History page to view a list of their previous orders. Each order entry displays the order ID, order date, number of items, total amount, and payment status. This page allows users to track their purchases and review order information after successful payment.

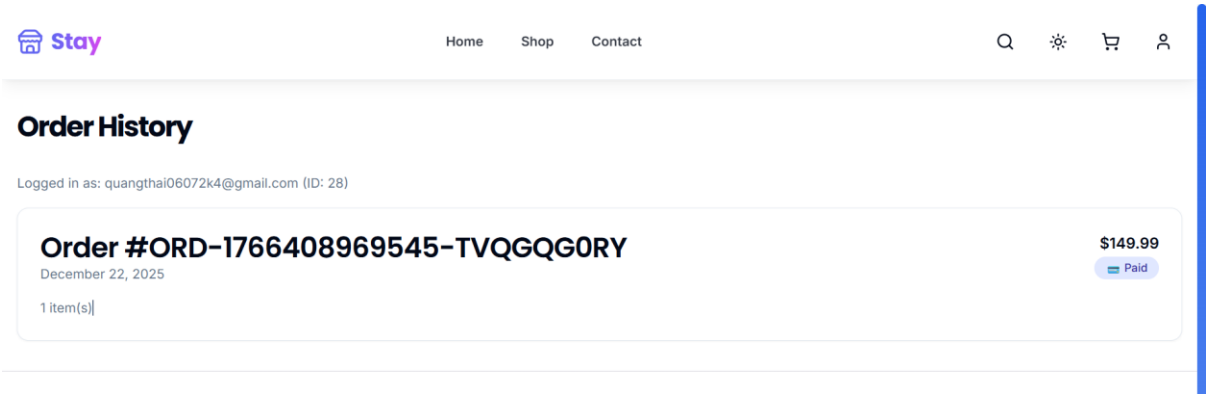

Figure 37:Order History Page

When users select an order from the Order History page, the system navigates to the Order Detail Page. This page displays detailed information about the selected order, including order items, quantity, price, and order summary. Users can view the shipping address, order status, payment status, and payment method. In addition, a support chat section is provided to allow users to contact staff regarding the selected order.

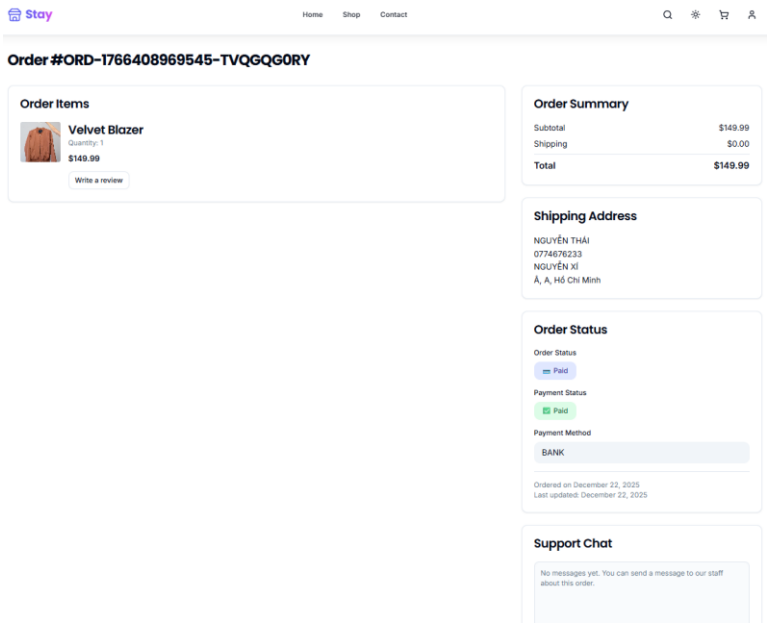

Figure 38: Order Detail Page

4. Order & System Management

4.1 User Account Management

From the user icon in the header, users can open the account menu to access key account-related features, including My Account, Orders, and Logout. This menu provides quick and convenient navigation for managing user information and viewing order history.

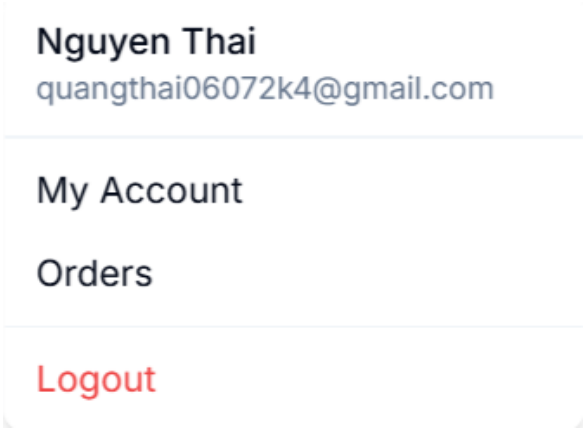

Figure 39: User Account Menu

After selecting My Account from the account menu, the system navigates to the My Account page. This page allows users to view and update their personal profile information, such as name and phone number, and optionally change their password. Users can save changes by clicking Update Profile.

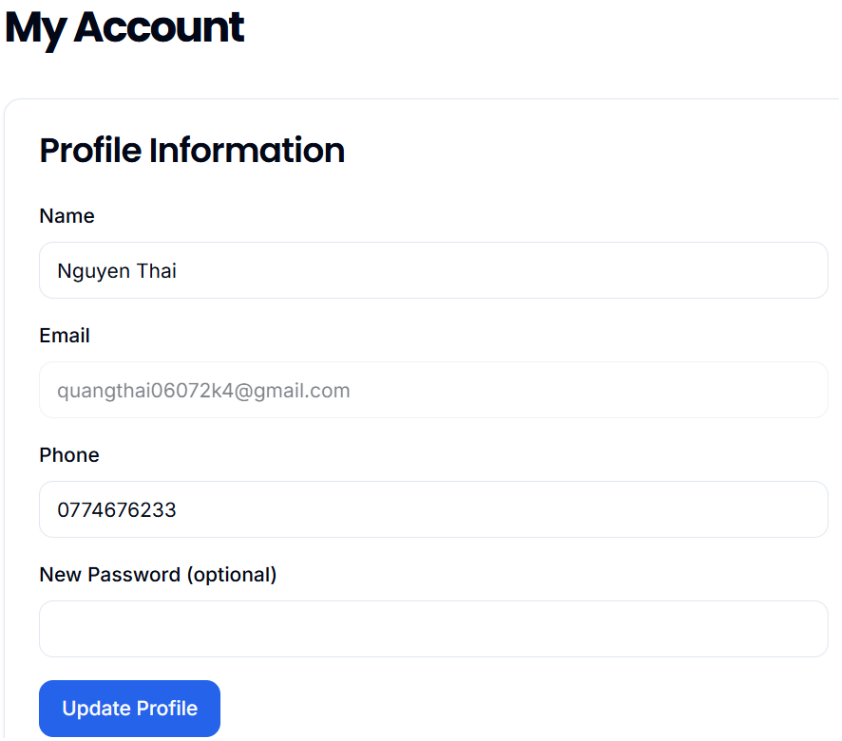

Figure 40: My Account Page

When users select Orders from the account menu, the system navigates to the Order History page. This page displays a list of all previous orders, allowing users to review order details, track order status, and access individual order information.

4.2 Admin / Staff Management

When an administrator logs into the system, the account menu displays additional options, including access to the Admin Panel. This menu allows authorized users to switch between user functionalities and administrative management features.

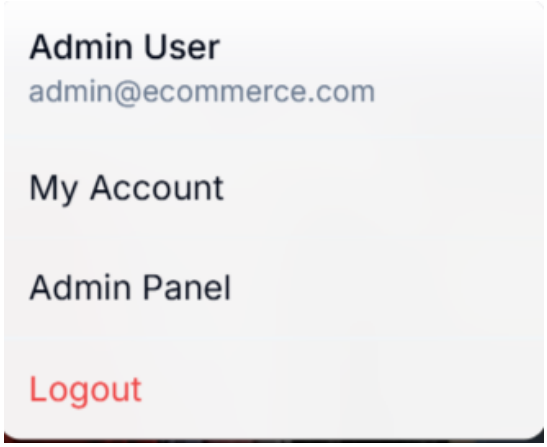

Figure 41: Admin Account Menu

The Admin Dashboard provides an overview of store activity and system statistics. It displays key metrics such as total orders, total revenue, total users, and total products, helping administrators monitor overall system performance.

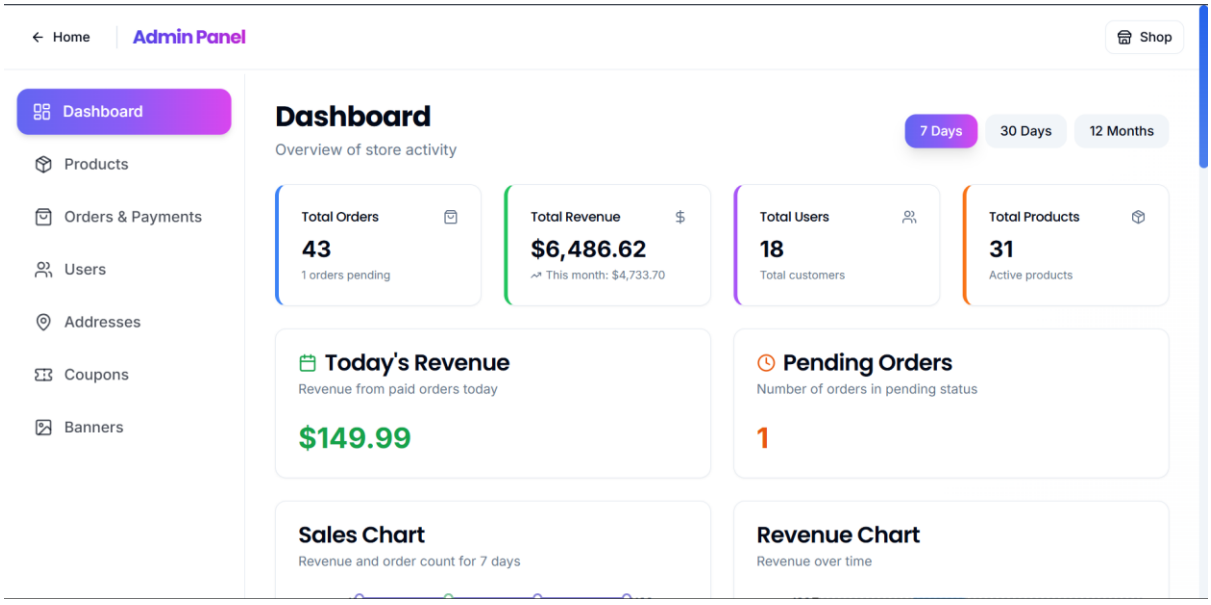

Figure 42: Admin Dashboard

From the admin panel, administrators can manage core system resources, including products, orders and payments, users, addresses, coupons, and banners. This centralized interface supports efficient store management and operational control.

The system implements role-based access control to distinguish between admin and staff users. Only users whose roles are assigned by an administrator can access management features through the Admin Panel or Staff Panel.

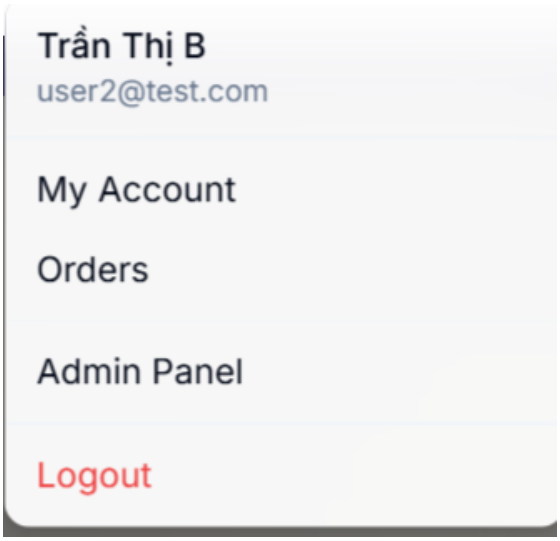

Figure 43: Staff Account Menu (Role-Based Access)

Staff users are granted access to a dedicated Staff Panel designed for daily operational tasks. This panel allows staff to monitor order statuses, manage orders and payments, view shipping addresses, update banners, and respond to customer inquiries via the support chat, without access to system-level configuration.

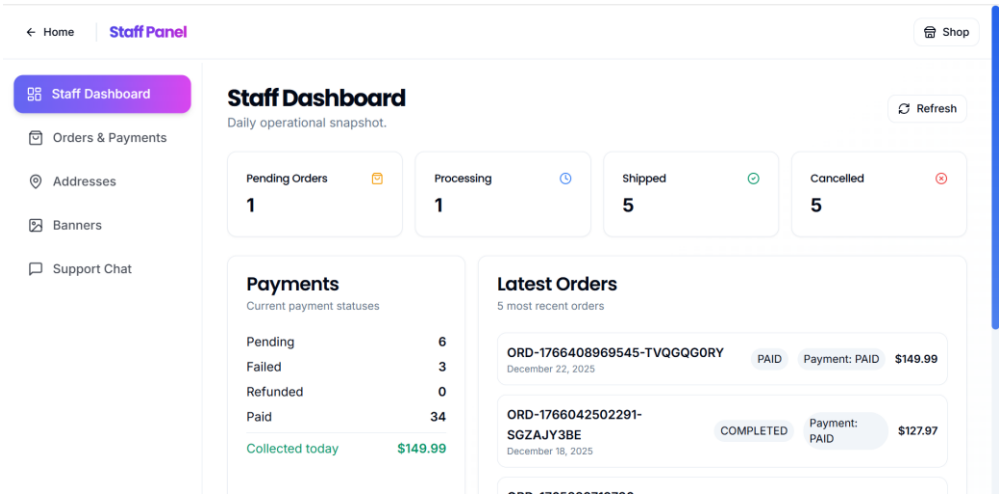

Figure 44: Staff Panel

This separation of responsibilities ensures secure system operation while supporting efficient store management.

IV. Discussion

This project successfully demonstrates the design and implementation of a full-stack e-commerce web application with a complete end-to-end workflow. The system

covers essential user functionalities, including authentication, product browsing, shopping cart management, checkout, payment simulation, and order tracking.

From a user perspective, the application provides a clear and intuitive purchasing flow, allowing users to browse products, place orders, and manage their accounts efficiently. The order history and order detail features enable users to track purchases and review order information after checkout, which is a core requirement for modern e-commerce platforms.

In addition to user-facing features, the system implements role-based access control to separate responsibilities between regular users, staff, and administrators. Staff members are provided with a dedicated panel for handling daily operations such as order processing and customer support, while administrators have access to higher-level management features. This separation improves system security and reflects real-world e-commerce practices.

The backend architecture follows a RESTful API design, with endpoints documented using Swagger. This approach improves maintainability, scalability, and ease of integration between the frontend and backend. Authentication and authorization mechanisms ensure that protected resources are accessed only by authorized users.

However, the system still has some limitations. Payment processing is currently simulated and does not integrate real payment gateways. Additionally, advanced features such as real-time notifications, automated shipment tracking, and advanced analytics are not fully implemented.

Overall, the project achieves its primary objectives and provides a solid foundation for future development. With further enhancements, the system can be extended into a production-ready e-commerce platform.

V. Conclusion

In this project, we designed and developed a full-stack e-commerce web application that demonstrates a complete purchasing workflow, from user authentication and product browsing to checkout, payment processing, and order management. The system successfully integrates frontend and backend components through a RESTful API architecture, ensuring clear separation of concerns and maintainability.

The application supports essential e-commerce functionalities for users, including shopping cart management, order history, and account management.

Additionally, role-based access control was implemented to distinguish between regular users, staff, and administrators, enabling secure system operation and efficient management of store activities.

Although payment processing is currently simulated and some advanced features are not fully implemented, the system fulfills its primary objectives and reflects real-world e-commerce design practices. Overall, this project provides a solid foundation for further enhancements and can be extended into a production-ready application with additional development

VI. References

[1] https://swagger.io/specification/
[2] https://nodejs.org/en/docs
[3] https://react.dev
[4] https://expressjs.com